

```

1  (*
2                                     CS51 Lab 12
3                               Imperative Programming and References
4  *)
5  (*
6                                     SOLUTION
7  *)
8
9  (*
10 Objective:
11
12 This lab provides practice with reference types and their use in
13 building mutable data structures and in imperative programming more
14 generally. It also gives further practice in using modules to
15 implement abstract data types.
16
17 There are three total parts to this lab. Please refer to the following
18 files to complete all exercises:
19
20     lab12_part1.ml -- Part 1: Fun with references (including `gensym`)
21 -> lab12_part2.ml -- Part 2: Appending mutable lists
22     lab12_part3.ml -- Part 3: Adding serialization to imperative queues
23 *)
24
25 (*=====
26 Part 2: Appending mutable lists
27
28 Recall the definition of the mutable list type from Section 15.4 of
29 the textbook: *)
30
31 type 'a mlist = 'a mlist_internal ref
32 and 'a mlist_internal =
33     | Nil
34     | Cons of 'a * 'a mlist ;;
35
36 (* Mutable lists are just like regular lists, except that each Nil or
37 cons is a *reference* to a mutable list, so that it can be updated. *)
38
39 (*.....
40 Exercise 5: Construct a function `mlist_empty : unit -> int mlist` that
41 returns an empty `mlist`.
42 *)
43
44 let mlist_empty () : int mlist =
45     ref Nil ;;
46
47 (* Now call the function `mlist_empty` within the REPL. You'll notice
48 that the value returned looks like a record structure with a single
49 `contents` field. That's because internally, OCaml implements `ref`
50 values as mutable records. *)
51
52 (*.....

```

```

53 Exercise 6: Construct a function `mlist_42 : unit -> int mlist` that
54 returns an `mlist` with the single integer element 42.
55 *)
56
57 let mlist_42 () : int mlist =
58   ref (Cons (42, ref Nil)) ;;
59
60 (* Now call the function `mlist_42` within the REPL. Again, you'll see
61 the telltale record structure with a single `contents` field. In the
62 future, you'll know why that is.
63
64 It's cumbersome to manually build up these mutable lists. Let's build
65 a function that allows us to generate mutable versions of regular
66 (immutable) lists. *)
67
68 (*.....
69 Exercise 7: Define a polymorphic function `mlist_of_list` that
70 converts a regular list to a mutable list, with behavior like this:
71
72   # let xs = mlist_of_list ["a"; "b"; "c"] ;;
73   val xs : string mlist =
74     {contents = Cons ("a",
75                       {contents = Cons ("b",
76                                         {contents = Cons ("c",
77                                                           {contents = Nil}})}))}}
78
79   # let ys = mlist_of_list [1; 2; 3] ;;
80   val ys : int mlist =
81     {contents = Cons (1,
82                       {contents = Cons (2,
83                                         {contents = Cons (3,
84                                                           {contents = Nil}})}))}}
85 .....*)
86
87 let rec mlist_of_list (lst : 'a list) : 'a mlist =
88   match lst with
89   | [] -> ref Nil
90   | hd :: tl -> ref (Cons (hd, mlist_of_list tl)) ;;
91
92 (*.....
93 Exercise 8: Define a function `mlength` to compute the length of an
94 `mlist`. Try to do this without looking at the solution that is given
95 in the book. (Don't worry about cycles...yet.)
96
97   # mlength (ref Nil) ;;
98   - : int = 0
99   # mlength (mlist_of_list [1; 2; 3; 4]) ;;
100   - : int = 4
101 .....*)
102
103 let rec mlength (mlst : 'a mlist) : int =
104   match !mlst with
105   | Nil -> 0
106   | Cons (_hd, tl) -> 1 + mlength tl ;;

```

```

107
108 (*.....
109 Exercise 9: What is the time complexity of the `mlength` function in
110 terms of the length of its list argument? Provide the tightest
111 complexity class, recorded using the technique from lab 10.
112 .....*)
113
114 type complexity =
115   | Unanswered
116   | Constant
117   | Logarithmic
118   | Linear
119   | LogLinear
120   | Quadratic
121   | Cubic
122   | Exponential ;;
123
124 (* ANSWER: The `length` function is linear in the length of its
125    argument:  $O(n)$ . *)
126 let length_complexity : complexity = Linear
127
128 (*.....
129 Exercise 10: Now, define a function `mappend` that takes a first
130 mutable list and a second mutable list and, as a side effect, causes
131 the first to *become* (as a side effect) the appending of the two
132 lists. A question to think about before you get started:
133
134     What is an appropriate return type for the `mappend` function?
135     (You can glean our intended answer from the examples below, but
136     try to think it through yourself first.)
137
138 Examples of use:
139
140     # let m1 = mlist_of_list [1; 2; 3] ;;
141     val m1 : int mlist =
142       {contents = Cons (1,
143         {contents = Cons (2,
144           {contents = Cons (3,
145             {contents = Nil}}))}})}
146
147     # let m2 = mlist_of_list [4; 5; 6] ;;
148     val m2 : int mlist =
149       {contents = Cons (4,
150         {contents = Cons (5,
151           {contents = Cons (6,
152             {contents = Nil}}))}})}
153
154     # mlength m1 ;;
155     - : int = 3
156
157     # mappend m1 m2 ;;
158     - : unit = ()
159
160     # mlength m1 ;;

```

```

161     - : int = 6
162
163     # m1 ;;
164     - : int mlist =
165     {contents = Cons (1,
166         {contents = Cons (2,
167             {contents = Cons (3,
168                 {contents = Cons (4,
169                     {contents = Cons (5,
170                         {contents = Cons (6,
171                             {contents = Nil}}}}}}}}}}}}
172     .....*)
173
174     (* Answer to thought question:
175
176     What is an appropriate return type for the `mappend` function?
177     (You can glean our intended answer from the examples below, but
178     try to think it through yourself first.)
179
180     ANSWER: Since `mappend` is called for its side effect, `unit` is
181     the appropriate return type.
182     *)
183
184     let rec mappend (xs : 'a mlist) (ys : 'a mlist) : unit =
185     match !xs with
186     | Nil -> xs := !ys
187     | Cons (_hd, tl) -> mappend tl ys ;;
188
189     (* What happens when you evaluate the following expressions
190     sequentially in order?
191
192     # let m = mlist_of_list [1; 2; 3] ;;
193     # mappend m m ;;
194     # m ;;
195     # mlength m ;;
196
197     Do you understand what's going on? *)

```